

(Class Template)

Template

(Generic Programming)

林致宇

Outline

- Function Template Review
- Template Parameters
- Class Template
- Template Specialization
- static_assert()
- Standard Template Library

class Template > 都是 Generic Type (泛型)
Function Template

Function Template, CH6 P66

Outline

- Function Template Review
- Template Parameters
- Class Template
- Template Specialization
- `static_assert()`
- Standard Template Library

Function Template

Reference:

<https://stackoverflow.com/questions/2023977/difference-of-keywords-typename-and-class-in-templates>

```
6  template<class T>
7  int min(T data[], int size) {
8      int index = 0;
9      for (int i = 1; i < size; i++) {
10         if (data[i] < data[index]) {
11             index = i;
12         }
13     }
14     return index;
15 }
16
17 int main() {
18     int idata[] = {68, 27, 32};
19     double ddata[] = {3.6, 7.2, 1.3};
20
21     cout << min<int>(idata, 3) << endl;
22     cout << min<double>(ddata, 3) << endl;
23 }
```

Outline

- Function Template Review
- Template Parameters
- Class Template
- Template Specialization
- `static_assert()`
- Standard Template Library

Nontype Template Parameter

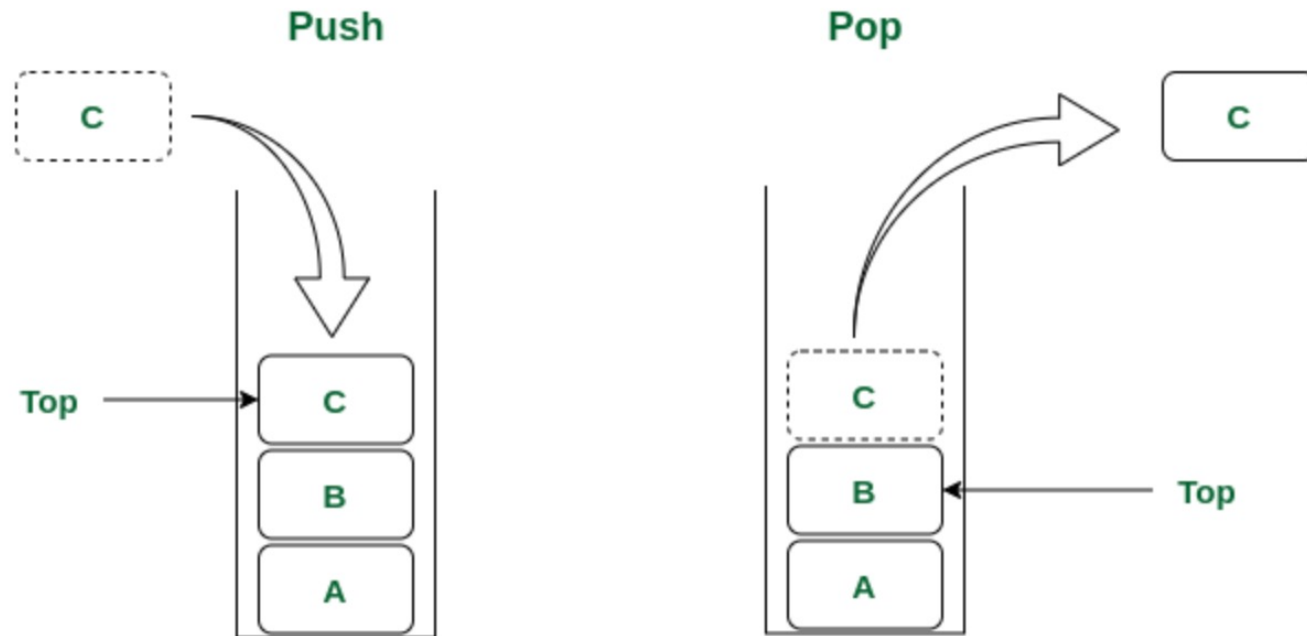
```
6  template<class T, int N> ← Template 也可以寫成非泛形的形式
7  int min(T data[N]) {
8      int index = 0;
9      for (int i = 1; i < N; i++) {
10         if (data[i] < data[index]) {
11             index = i;
12         }
13     }
14     return index;
15 }
16
17 int main() {
18     int idata[3] = {68, 27, 32};
19     double ddata[3] = {3.6, 7.2, 1.3};
20
21     cout << min<int, 3>(idata) << endl;
22     cout << min<double, 3>(ddata) << endl;
23 }
```

Outline

- Function Template Review
- Template Parameters
- Class Template *vector 其实就是一个 Class Template*
- Template Specialization
- static_assert()
- Standard Template Library

Stack

- LIFO (Last In First Out) / FILO (First In Last Out)



Stack Data Structure

Reference: <https://www.geeksforgeeks.org/stack-data-structure/>

Applications of Stack

- Function calls and recursion (Call Stack)
- File System
- Undo/Redo operations
- Browser History

Stack Class

```
5  template<class T, int SIZE = 3>
6  class Stack {
7  private:
8      int count;
9      T buffer[SIZE];
10 public:
11     Stack() { count = 0; };
12     int push(T data);
13     T pop();
14 };
15
16 template<class T, int SIZE>
17 int Stack<T, SIZE>::push(T data) {
18     if (count == SIZE) {
19         return -1;
20     } else {
21         buffer[count++] = data;
22     }
23     return count;
24 }
```

可能会考! <

记得这裡!

```
26 template<class T, int SIZE>
27 T Stack<T, SIZE>::pop() {
28     if (count == 0) {
29         throw -1;
30     }
31     return buffer[--count];
32 }
```

Stack Class

```
34  int main() {
35      Stack<int,2> stack1;
36      cout << stack1.push(1) << endl;
37      cout << stack1.push(2) << endl;
38      cout << stack1.push(3) << endl;
39      do {
40          try {
41              auto result = stack1.pop();
42              cout << result << endl;
43          } catch (int e) {
44              cout << "Empty!" << endl;
45              break;
46          }
47      } while (true);
48  }
```

Stack Class

```
34 class Car {
35 public:
36     int id;
37 };
38
39 int main() {
40     Car car1; car1.id = 1;
41     Car car2; car2.id = 2;
42     Car car3; car3.id = 3;
43
44     Stack<Car,2> stack1;
45     cout << stack1.push(car1) << endl;
46     cout << stack1.push(car2) << endl;
47     cout << stack1.push(car3) << endl;
48     do {
49         try {
50             Car result = stack1.pop();
51             cout << result.id << endl;
52         } catch (int e) {
53             cout << "Empty!" << endl;
54             break;
55         }
56     } while (true);
57 }
```

Outline

- Function Template Review
- Template Parameters
- Class Template
- **Template Specialization**
- `static_assert()`
- Standard Template Library

Template Specialization

```
5  template<class T>
6  class Equal {
7  private:
8  public:
9      bool check(T data1, T data2) {
10         if (data1 == data2) {
11             return true;
12         } else {
13             return false;
14         }
15     }
16 };
```

```
18 int main() {
19     cout << boolalpha;
20     int i1 = 6, i2 = 8, i3 = 8;
21     Equal<int> e1;
22     cout << e1.check(i1, i2) << endl;
23     cout << e1.check(i2, i3) << endl;
24     Equal<double> e2;
25     cout << e2.check(5.0, 3.0) << endl;
26     cout << e2.check(5.0, 5.0) << endl;
27     Equal<int *> e3;
28     cout << e3.check(&i1, &i2) << endl;
29     cout << e3.check(&i2, &i3) << endl;
30 }
```

```
[lincyu@Chih-YudeMacBook-Pro Chapter15 % ./a.out
false
true
false
true
false
false]
```

Template Specialization

会考

```
18  template<>
19  class Equal<int *> {
20  private:
21  public:
22      bool check(int * data1, int * data2) {
23          if (*data1 == *data2) {
24              return true;
25          } else {
26              return false;
27          }
28      }
29  };

lincyu@Chih-YudeMacBook-Pro Chapter15 % ./a.out
false
true
false
true
false
true
```

Template Specialization (Function)

```
5  template<typename T>
6  bool isEqual(T data1, T data2) {
7      if (data1 == data2) {
8          return true;
9      }
10     return false;
11 }
12 template<>
13 bool isEqual<char *>(char * data1, char * data2) {
14     if (strcmp(data1, data2) == 0 ) {
15         return true;
16     }
17     return false;
18 }
19
20 int main() {
21     cout << boolalpha;
22     char * str1 = "Hello";
23     char * str2 = "Hello";
24     cout << isEqual(str1, str2) << endl;
25 }
```


Outline

- Function Template Review
- Template Parameters
- Class Template
- Template Specialization
- `static_assert()`
- Standard Template Library

static_assert() (The example is for C++17)

```
1  #include<iostream>
2  #include<type_traits>
3  using namespace std;
4
5  template<class T>
6  class Equal {
7  private:
8  public:
9      static_assert(!is_pointer_v<T>);
10     bool check(T data1, T data2) {
11         if (data1 == data2) {
12             return true;
13         } else {
14             return false;
15         }
16     }
17 };
```

```
19 int main() {
20     cout << boolalpha;
21     int i1 = 6, i2 = 8, i3 = 8;
22     Equal<int> e1;
23     cout << e1.check(i1, i2) << endl;
24     cout << e1.check(i2, i3) << endl;
25     Equal<double> e2;
26     cout << e2.check(5.0, 3.0) << endl;
27     cout << e2.check(5.0, 5.0) << endl;
28     // Equal<int *> e3; // Compiler Error
29     // cout << e3.check(&i1, &i2) << endl;
30     // cout << e3.check(&i2, &i3) << endl;
31 }
```

Outline

- Function Template Review
- Template Parameters
- Class Template
- Template Specialization
- `static_assert()`
- Standard Template Library

Standard Template Library

- `<algorithm>`
- `<stack>`

<algorithm>

```
1  #include<iostream>
2  #include<algorithm>
3
4  int main() {
5      int arr1[] = {30, 27, 19, 56, 72, 36};
6      double arr2[] = {3.6, 7.2, 8.5, 1.6, 9.1, 5.6};
7
8      std::cout << *std::min_element<int *>(&arr1[0], &arr1[5]) << std::endl;
9      std::cout << *std::min_element<double *>(&arr2[0], &arr2[5]) << std::endl;
10 }
```

<stack>

```
1  #include<iostream>
2  #include<stack>
3  using namespace std;
4
5  int main() {
6      stack<int> s1;
7      stack<double> s2;
8
9      s1.push(5);
10     s1.push(6);
11     cout << s1.top() << endl;
12     s1.pop();
13     cout << s1.top() << endl;
14     s1.pop();
15
16     s2.push(3.5);
17     cout << s2.top() << endl;
18     s2.pop();
19 }
```